

directory as the `.py` file we're writing, the filename is all that `Path` needs to access the file.

NOTE

VS Code looks for files in the folder that was most recently opened. If you're using VS Code, start by opening the folder where you're storing this chapter's programs. For example, if you're saving your program files in a folder called `chapter_10`, press `CTRL-O` (⌘-O on macOS), and open that folder.

Once we have a `Path` object representing `pi_digits.txt`, we use the `read_text()` method to read the entire contents of the file ❷. The contents of the file are returned as a single string, which we assign to the variable `contents`. When we print the value of `contents`, we see the entire contents of the text file:

```
3.1415926535
8979323846
2643383279
```

The only difference between this output and the original file is the extra blank line at the end of the output. The blank line appears because `read_text()` returns an empty string when it reaches the end of the file; this empty string shows up as a blank line.

We can remove the extra blank line by using `rstrip()` on the `contents` string:

```
from pathlib import Path

path = Path('pi_digits.txt')
contents = path.read_text()
contents = contents.rstrip()
print(contents)
```

Recall from Chapter 2 that Python's `rstrip()` method removes, or strips, any whitespace characters from the right side of a string. Now the output matches the contents of the original file exactly:

```
3.1415926535
8979323846
2643383279
```

We can strip the trailing newline character when we read the contents of the file, by applying the `rstrip()` method immediately after calling `read_text()`:

```
contents = path.read_text().rstrip()
```

This line tells Python to call the `read_text()` method on the file we're working with. Then it applies the `rstrip()` method to the string that `read_text()` returns. The cleaned-up string is then assigned to the variable `contents`. This approach is called *method chaining*, and you'll see it used often in programming.